

DeskMind AI

PROJECT STATUS & SEMESTER ROADMAP

A local-first desktop intelligence platform

Prepared as an evidence-based project report for the semester prototype. It separates work that is implemented today from work that remains partial, planned, or part of the long-term vision.

Semester objective	Deliver a stable, extensible desktop assistant prototype - not a full J.A.R.V.I.S.-style system.
Current focus	Connect the end-to-end Brain -> Dispatcher -> Registry -> Capability flow, then demonstrate useful local capabilities.
Architecture principle	The Brain decides; independent capabilities execute. Gemini, local models, tools, and databases are replaceable implementation resources.

Status date: semester planning baseline | Prepared from the project research record and referenced commits

1. Executive summary

DeskMind AI is evolving from a desktop chat application into a local-first, extensible intelligence platform. The foundation is credible: a PySide6 chat interface, responsive threaded Gemini requests, a basic Brain, an abstract Capability contract, and a lazy-loaded capability registry have been implemented and tested. The most important remaining engineering work is integration: the interface must stop communicating directly with Gemini and instead route requests through the Brain, Dispatcher, Registry, and a capability.

The semester should prioritize a demonstrable architectural prototype with Chat, PDF, Notes, Reminder, File Search, and local SQLite-backed data. Voice, a sophisticated memory system, local LLM orchestration, cloud scaling, and any proprietary model training remain future work.

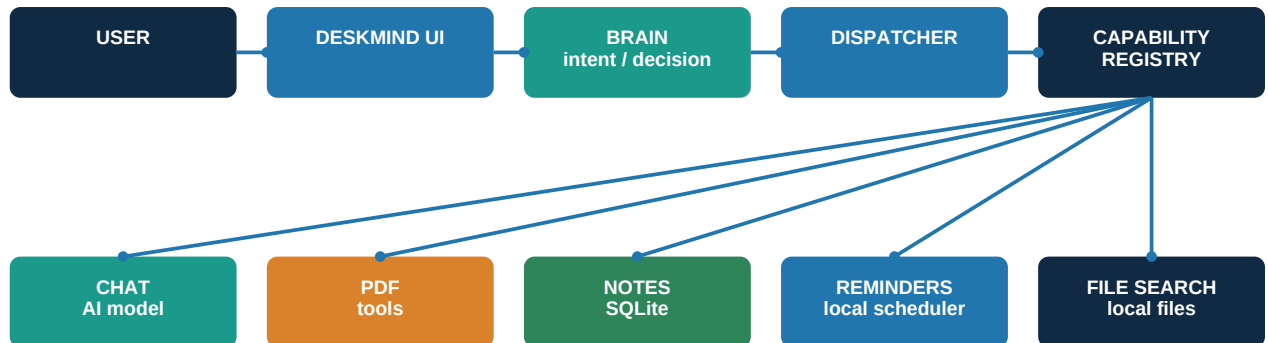
Current status inventory

Area	Status	Evidence / scope
Project foundation	Implemented	Repository established; baseline commit 9d7e0c8.
PySide6 chat interface	Partial / mostly implemented	Input, bubbles, scrolling, auto-scroll fix, and threaded execution are working.
Gemini integration	Implemented	Newer Google GenAI approach tested; 429 quota exhaustion observed.
Brain	Basic implementation	Central request entry point exists; needs complete routing path.
Capability abstraction	Implemented	Common execute(request, context) contract established.
Lazy capability registry	Implemented	Factories + instances, lazy singleton creation, custom missing-capability error; commit 4e46d58.
Chat capability	Prototype	Implements common interface; requires finished Gemini-backed flow.
Dispatcher + intent detection	Planned / incomplete	Dispatcher was recorded as empty; must connect intent to a registered capability.
PDF, Notes, Reminder, File Search	Planned	Semester capabilities, not yet claimed as implemented.
Local storage / memory	Planned	SQLite is the intended semester storage choice; memory system is future-facing.

Status key: Implemented = verified foundation; Partial/Prototype = exists but requires integration or hardening; Planned = approved semester scope, not built; Long-term = outside this semester.

2. Target architecture and request flow

The target separates presentation, decision-making, dispatch, and execution. This keeps the desktop UI independent from a particular model provider and allows capabilities to be added without rewriting the core.



The UI requests outcomes; the Brain selects a capability; capabilities invoke the appropriate model, tool, or local data source.

Operational request flow

Step	Responsibility	Semester expectation
1. UI	Collect a user request and present progress/results.	UI calls Brain.process(); it does not import Gemini directly.
2. Brain	Own the application-level decision path.	Obtain intent and coordinate error handling/fallbacks.
3. Dispatcher	Map intent to a capability name.	Validate route and request the registry instance.
4. Registry	Lazily create and cache a capability.	Return the registered capability or raise a meaningful error.
5. Capability	Execute one focused task.	Invoke Gemini, a PDF tool, SQLite, scheduler, or file search as appropriate.
6. Result	Return a structured outcome to the UI.	Show result, user-safe failure message, and logging context.

Design boundary: DeskMind is not Gemini. Gemini, a local LLM, OCR, file processors, databases, and operating-system tools are resources selected behind the capability boundary.

3. Completed work and key commits

The following are recorded accomplishments. They are intentionally described at their demonstrated maturity level rather than promoted to finished product features.

Key commits

Commit	Recorded purpose	Why it matters
9d7e0c8	adding project from new branch	Established the project baseline and repository structure.
4e46d58	feat: add plugin-based capability registry	Introduced the capability architecture: registration, lazy construction/reuse, and an explicit missing-capability failure path.

Implemented foundation

- PySide6 desktop chat with message entry, message bubbles, scrolling, and tested automatic scroll-to-latest behavior.
- Gemini API integration using the newer Google GenAI direction; successful responses were tested.
- Worker-based Gemini execution so slower AI requests do not freeze the UI.
- Basic Brain entry point and an abstract Capability interface.
- Capability registry with factory registration, lazy object construction/reuse, and CapabilityNotFoundError.
- ChatCapability prototype and a basic registry-to-capability test (recorded output: "Chat capability Executed").

Important engineering lesson

The registry commit was temporarily removed from the active branch through a hard reset and recovered using Git reflog. The recovery reinforced a simple project practice: test, commit, push, then review; avoid destructive Git actions without a recovery plan.

4. Outstanding semester work

The work below is prioritized for an achievable prototype. Dependencies are explicit so the team can avoid building features on an unconnected core.

Priority	Deliverable	Dependency / acceptance evidence
P0	End-to-end Chat flow	Finish intent + Dispatcher; UI -> Brain -> Dispatcher -> Registry -> ChatCapability -> Gemini. Demonstrate no direct Gemini dependency in UI.
P0	Error handling and logging	Structured errors for quota, network, missing capability, and invalid request; user-safe UI messages.
P1	SQLite foundation	Schema and data-access boundary for notes, reminders, and settings; local-only storage.
P1	Notes capability	Create, read, update, delete notes through the full request flow.
P1	PDF capability	Process user-selected PDFs with clear limits, error handling, and test fixtures.
P1	Reminder capability	Create/list/cancel basic local reminders; demonstrate reliable persistence.
P1	File search capability	Search only user-approved local paths; return useful results without broad OS access.
P2	Test suite + CI	Unit tests for registry, dispatcher, Brain, chat, PDF, memory/storage; GitHub Actions when baseline is stable.
P2	Voice proof of concept	Optional; only after typed workflow is stable.

Dependency chain: registry (implemented) -> dispatcher/intent -> Brain flow -> chat integration -> reusable storage/security/error patterns -> additional capabilities -> test/CI hardening.

5. Risks and mitigations

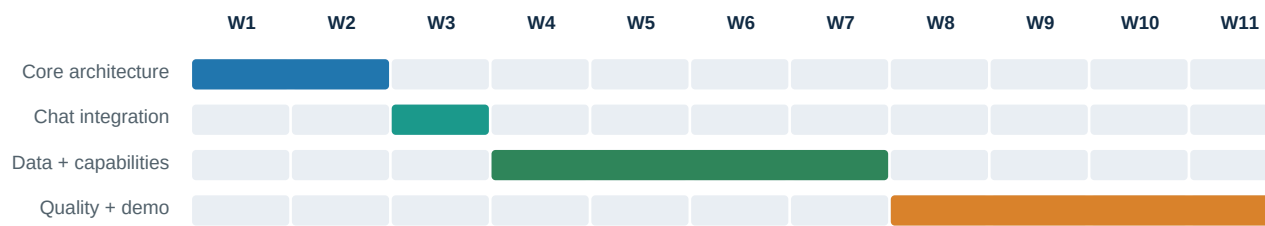
Risk	Impact	Mitigation
Gemini free-tier quota / 429 RESOURCE_EXHAUSTED	Chat unavailable or unreliable during a demo.	Graceful retry/message; avoid sending every operation to an LLM; plan a provider/local-model fallback later.
Direct UI-to-Gemini coupling persists	Architecture cannot scale to multiple capabilities.	Make Brain the only UI entry point; verify imports and end-to-end tests.
Scope grows into “full J.A.R.V.I.S.”	Core prototype is incomplete by semester end.	Treat voice, full memory, cloud/enterprise, and model training as long-term scope.
Sensitive data or keys exposed	Security and trust failure.	Use .env/secure configuration, keep secrets out of Git, validate inputs, enforce controlled file access.
File/OS capability risk	Accidental broad access or unsafe operations.	User-selected paths, permission boundaries, allowlists, and clear confirmation flows.
Insufficient automated tests	Regressions as new capabilities are added.	Build focused unit tests alongside every P0/P1 module; add integration smoke tests.
Git history mistakes	Loss of working progress.	Commit logical milestones, push, use reflog awareness, and avoid destructive reset commands.

Privacy posture

The semester prototype should use a local-first model: one user, one device, SQLite-backed application data. PostgreSQL is more appropriate later for authenticated multi-user or cloud deployment.

6. Realistic 12-week roadmap

This roadmap assumes the existing foundation is available at Week 1. The goal is a stable demonstrator, with a buffer for testing and integration rather than an overextended feature list.



Planning horizon: adjust calendar dates to the actual semester schedule.

Week	Primary goal	Concrete output
1	Close architecture gaps	Define request/result contracts; complete Dispatcher and intent routing tests.
2	First end-to-end route	Brain -> Dispatcher -> Registry -> ChatCapability path passing a smoke test.
3	Decouple UI	UI invokes Brain; threaded execution and quota/network errors remain responsive.
4	Storage base	SQLite schema, repository/data-access layer, configuration and logging.
5	Notes MVP	Notes CRUD capability demonstrated through UI.
6	PDF MVP	User-selected PDF processing with fixture tests and safe failures.
7	Reminder + file search MVP	Local reminders plus scoped file-search capability.
8	Integration hardening	Cross-capability tests, error/recovery paths, regression pass.
9	Security and privacy review	Secrets, config, validation, file permissions, and safe plugin registration.
10	UX and reliability pass	Loading/error states, logs, packaging/run instructions.
11	Demo rehearsal	Professor/team demonstration script, screenshots, and final bug fixes.
12	Semester handoff	Release tag/commit, report, test results, and next-semester backlog.

7. Next-session developer checklist

Use this sequence to resume development without losing the architectural thread.

1. Check git status and confirm the intended branch/revision before editing.
2. Review the current UI, Brain, Dispatcher, Registry, and ChatCapability together.
3. Write/repair Dispatcher and intent routing before adding a new capability.
4. Define a minimal request/result shape and error contract for the end-to-end chat path.
5. Change UI code so it talks only to Brain, not Gemini-specific services/workers.
6. Run a manual smoke test: send chat -> route -> registry -> capability -> result.
7. Add or update unit tests for every changed core module.
8. Move API keys to secure configuration; verify they are not tracked by Git.
9. Commit a logical milestone with a clear message, push it, and record the result.

Semester-end success criteria

The project is successful when the following can be demonstrated reliably to a professor/team:

- A PySide6 desktop interface remains responsive while handling AI/tool work.
- A typed user request travels through UI -> Brain -> Dispatcher -> Registry -> Capability; the UI is not directly coupled to Gemini.
- At least Chat plus several useful scoped capabilities (target: Notes, PDF, Reminder, File Search) work with local-first data where applicable.
- Failures such as missing capabilities, quota exhaustion, unavailable network/model, and bad files yield controlled, understandable responses rather than crashes.
- Core architecture and selected capabilities have unit/integration evidence, documented setup, and a reproducible demo path.
- Secrets are excluded from version control and local file access is deliberately constrained.

8. Long-term vision

After the semester prototype proves the architecture, DeskMind can expand into a richer local intelligence platform. These are intentional future directions, not completed semester deliverables.

Long-term direction	Reason to defer
Memory manager: short-term context, preferences, user facts, projects, and history	Needs careful privacy design, retrieval evaluation, and a stable storage boundary first.
AI manager with Gemini, local LLMs (for example via Ollama/Llama), and model selection	Requires robust fallbacks, performance work, and real usage data.
Voice interaction: speech recognition -> Brain -> capability -> text-to-speech	A basic proof of concept may be optional; polished voice UX is outside core scope.
Cloud / multi-user deployment with authentication and PostgreSQL	The current prototype is one user on one device; SQLite remains the practical semester choice.
A proprietary "DeskMind Brain" model	Training a competitive LLM needs large datasets, GPU infrastructure, evaluation, safety work, and substantial resources.
Broader plugins and controlled OS automation	Must follow a proven permission model and strong safety boundaries.

Closing position

DeskMind's strength will be its architecture: a system that can add useful intelligence safely and incrementally. A complete, well-tested prototype that demonstrates the core decision and capability pipeline is a stronger semester outcome than an overpromised assistant with disconnected features.